Solution: Lowest Common Ancestor of a Binary Search Tree

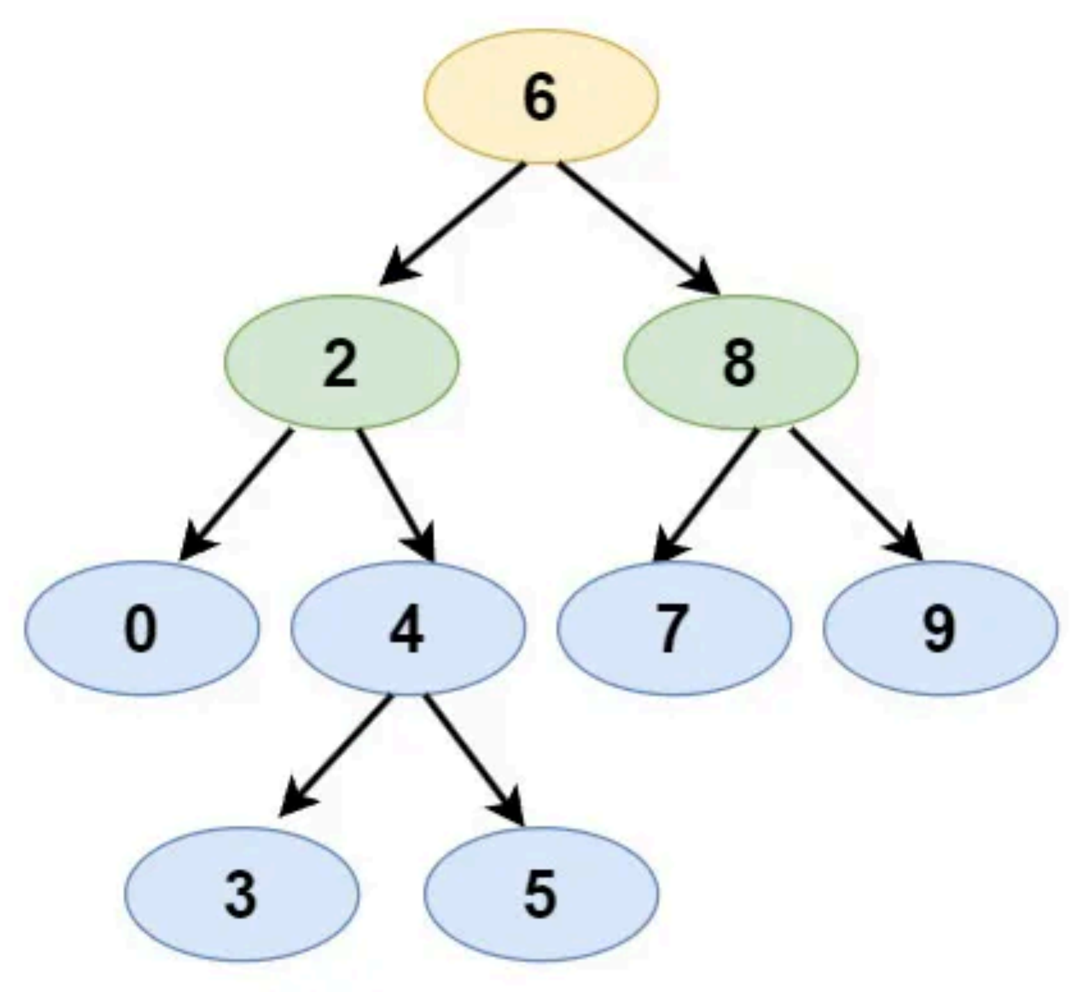
Problem Statement

Given a binary search tree (BST) and two of its nodes, find the node that is the lowest common ancestor (LCA) of the two given nodes. The LCA of two nodes is the node that lies in between the two nodes in terms of value and is the furthest from the root. In other words, it's the deepest node where the two nodes diverge in the tree. Remember, in a BST, nodes have unique values.

Examples

1.

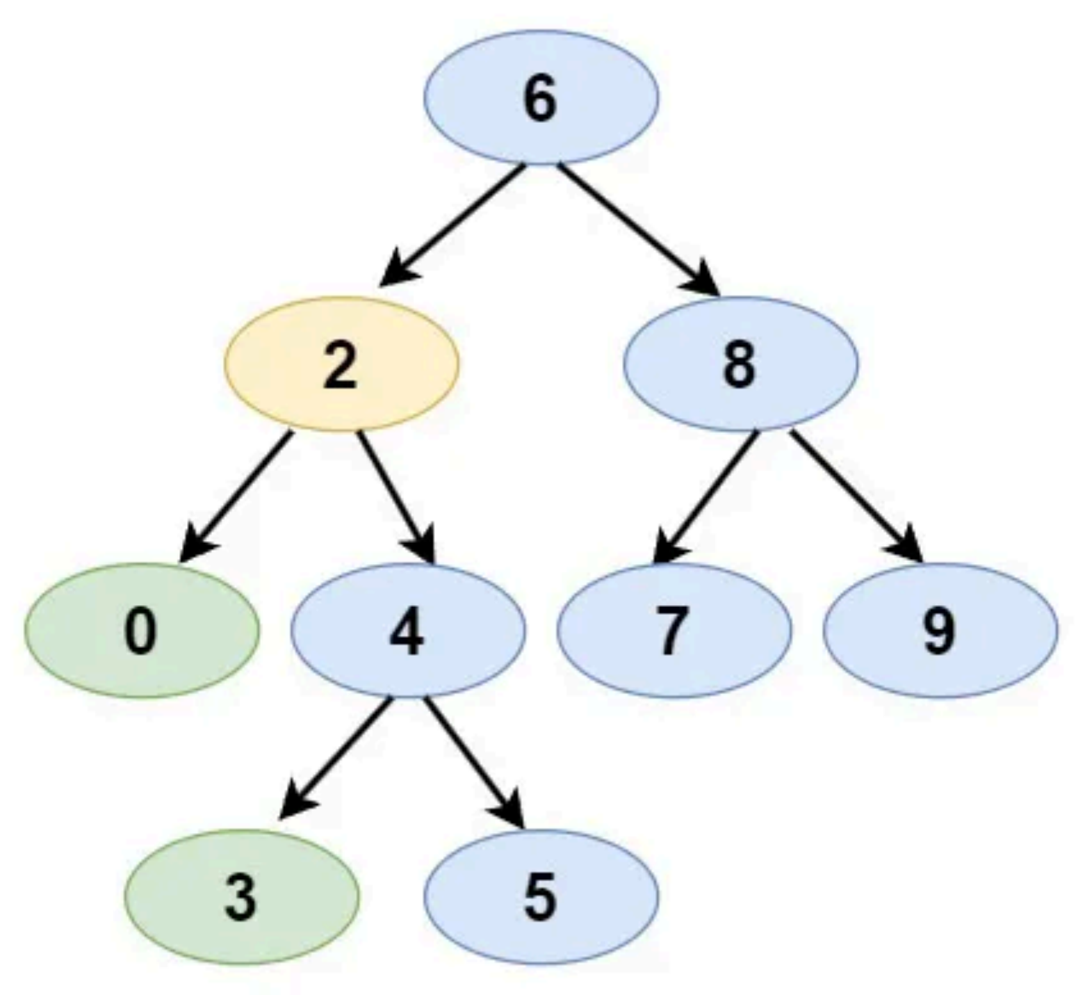
- Input:
  - BST: [6,2,8,0,4,7,9,null,null,3,5]
  - Node 1: 2
  - Node 2: 8
- Expected Output: 6



- Justification: The nodes 2 and 8 are on the left and right children of node 6. Hence, node 6 is their LCA.

2.

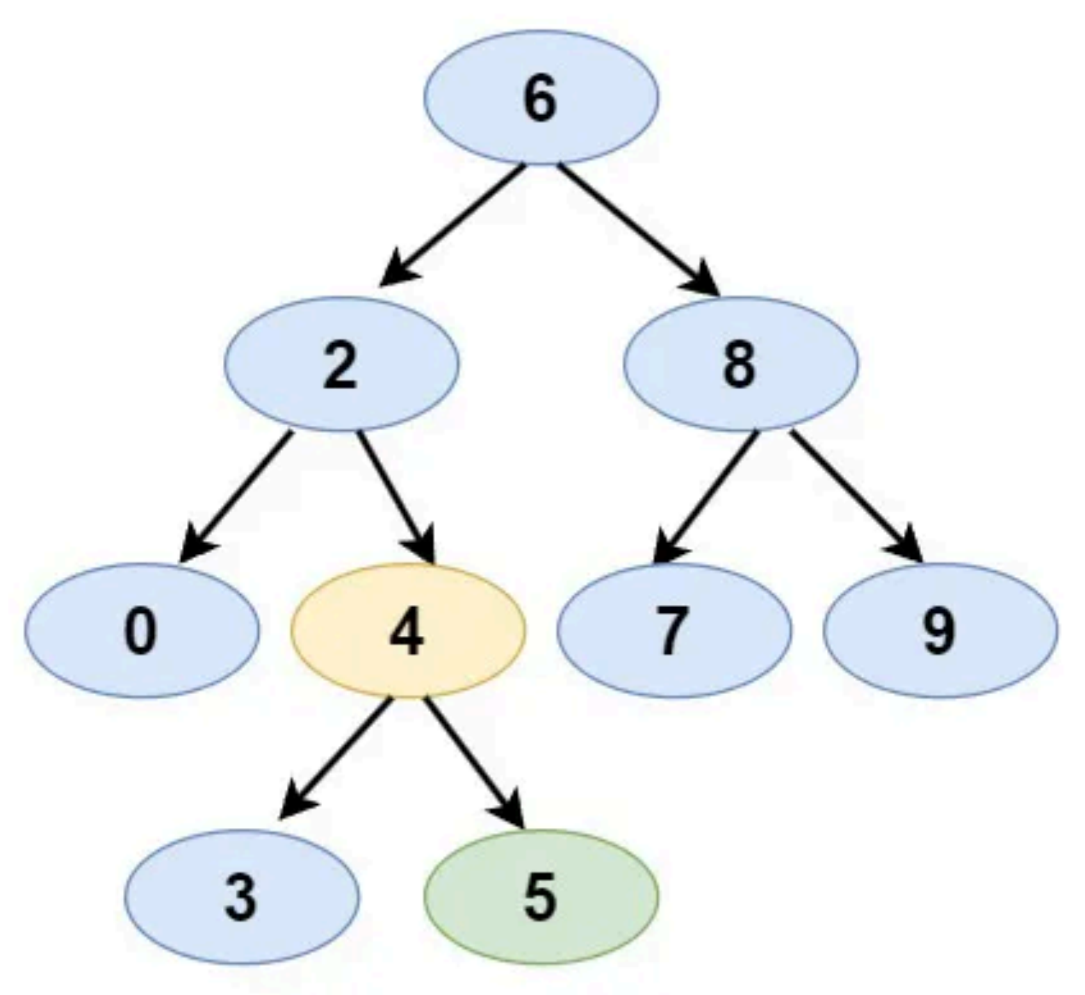
- Input:
  - BST: [6,2,8,0,4,7,9,null,null,3,5]
  - Node 1: 0
  - Node 2: 3
- Expected Output: 2



- Justification: The nodes 0 and 3 are on the left and right children of node 2, which is the closest ancestor to these nodes.

3.

- Input:
  - BST: [6,2,8,0,4,7,9,null,null,3,5]
  - Node 1: 4
  - Node 2: 5
- Expected Output: 4



- Justification: Node 5 is the right child of node 4. Hence, the LCA is node 4 itself.

Solution

The binary search tree property helps us find the solution without exploring the entire tree. Each node, starting from the root, provides a range based on its value that can determine where the two nodes are located.

1. **Starting at the Root**: Begin at the root of the BST.
2. **Determining Direction**: If the values of both nodes are greater than the current root node's value, then the LCA must be in the right subtree. If the values are both less, then the LCA is in the left subtree.
3. **Divergence Point**: If one node's value is less than the root's value and the other node's value is greater (or if one of them matches the root's value), then the current root is the LCA, since the path to reach both nodes diverges from here.
4. **Iterative Search**: Repeat the process iteratively on the selected subtree (either left or right) until you find the LCA.

Algorithm Walkthrough

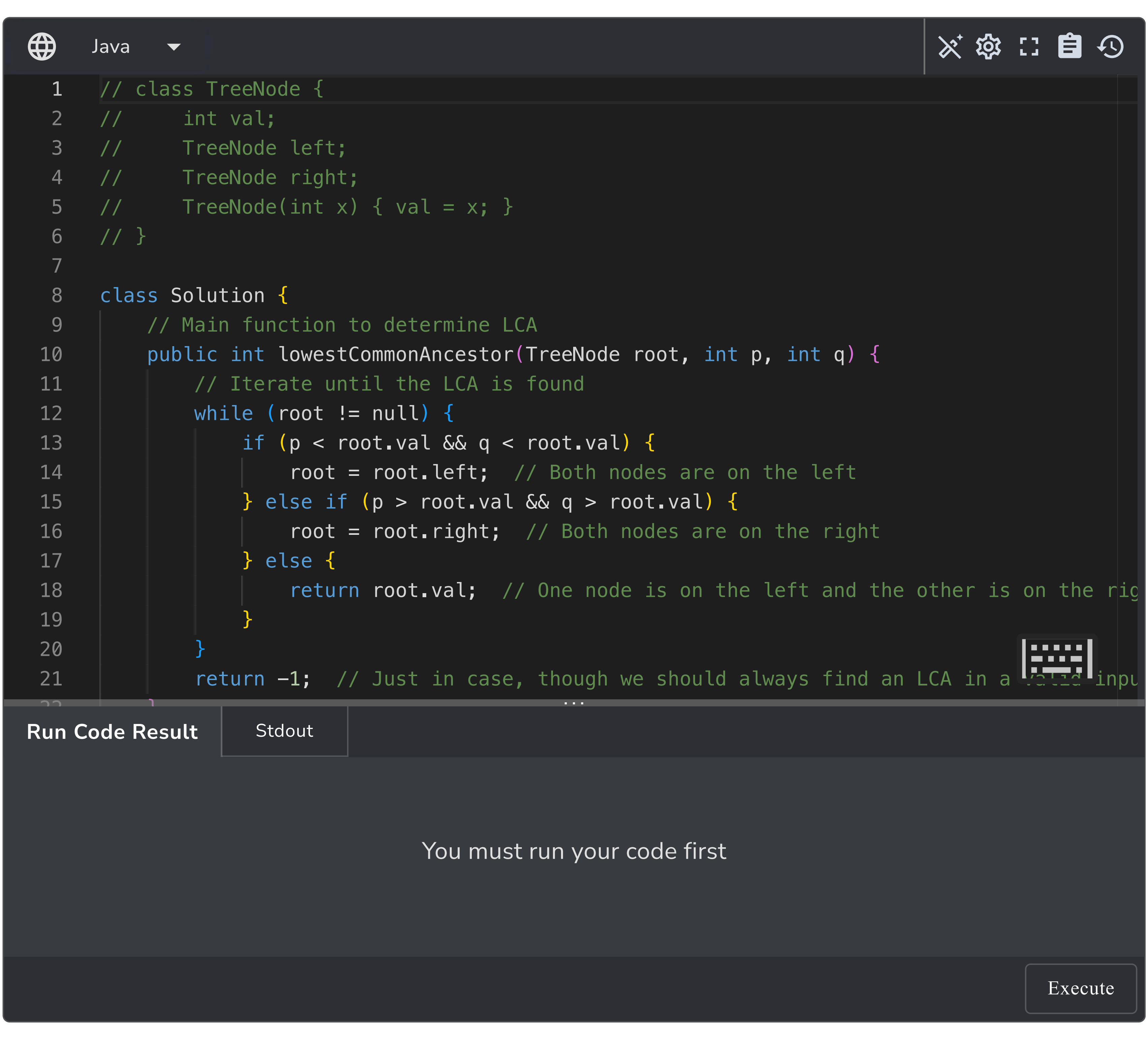
Using the first example input:

- BST: [6,2,8,0,4,7,9,null,null,3,5]
- Node 1: 2
- Node 2: 8

Steps:

- Start at root which is 6.
- Both 2 and 8 are on different sides of 6 (2 on the left and 8 on the right).
- Therefore, 6 is the lowest common ancestor.

Code



```java
// class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode(int x) { val = x; }
// }

class Solution {
    // Main function to determine LCA
    public int lowestCommonAncestor(TreeNode root, int p, int q) {
        // Iterate until the LCA is found
        while (root != null) {
            if (p < root.val && q < root.val) {
                root = root.left;  // Both nodes are on the left
            } else if (p > root.val && q > root.val) {
                root = root.right;  // Both nodes are on the right
            } else {
                return root.val;  // One node is on the left and the other is on the rig
            }
        }
        return -1;  // Just in case, though we should always find an LCA in a valid inpu
```

You must run your code first

Complexity Analysis

The algorithm traverses the BST in a single path, either going left or right, but never both. Therefore:

- **Time Complexity**: O(h), where h is the height of the BST.
- **pace Complexity**: O(1) since we only used a constant amount of space.